



软件架构设计

主讲：李伟刚

liweigang@nwpu.edu.cn

西北工业大学软件学院



第十章 并发和实时软件架构设计

主要内容

- 10.1 概述
- 10.2 实时软件架构中的控制模式
- 10.3 并发任务
- 10.4 I/O任务组织准则
- 10.5 内部任务组织准则
- 10.6 开发并发任务架构
- 10.7 任务间的通信和同步
- 10.8 任务接口规约



10.1 概述

- **实时软件架构**是指必须同时处理多个输入事件流的并发式架构
- 通常与状态相关，具有集中或非集中的控制方式
- 设计**并发对象**（并发任务）是设计实时软件架构的重要工作
- 实时软件架构通常是分布式的，可看作基于构件的软件架构的特例
- 设计并发软件时，将系统分解成一系列并发任务，并定义了任务间的接口及交互
- 客户端和服务各自都可以设计成并发软件架构



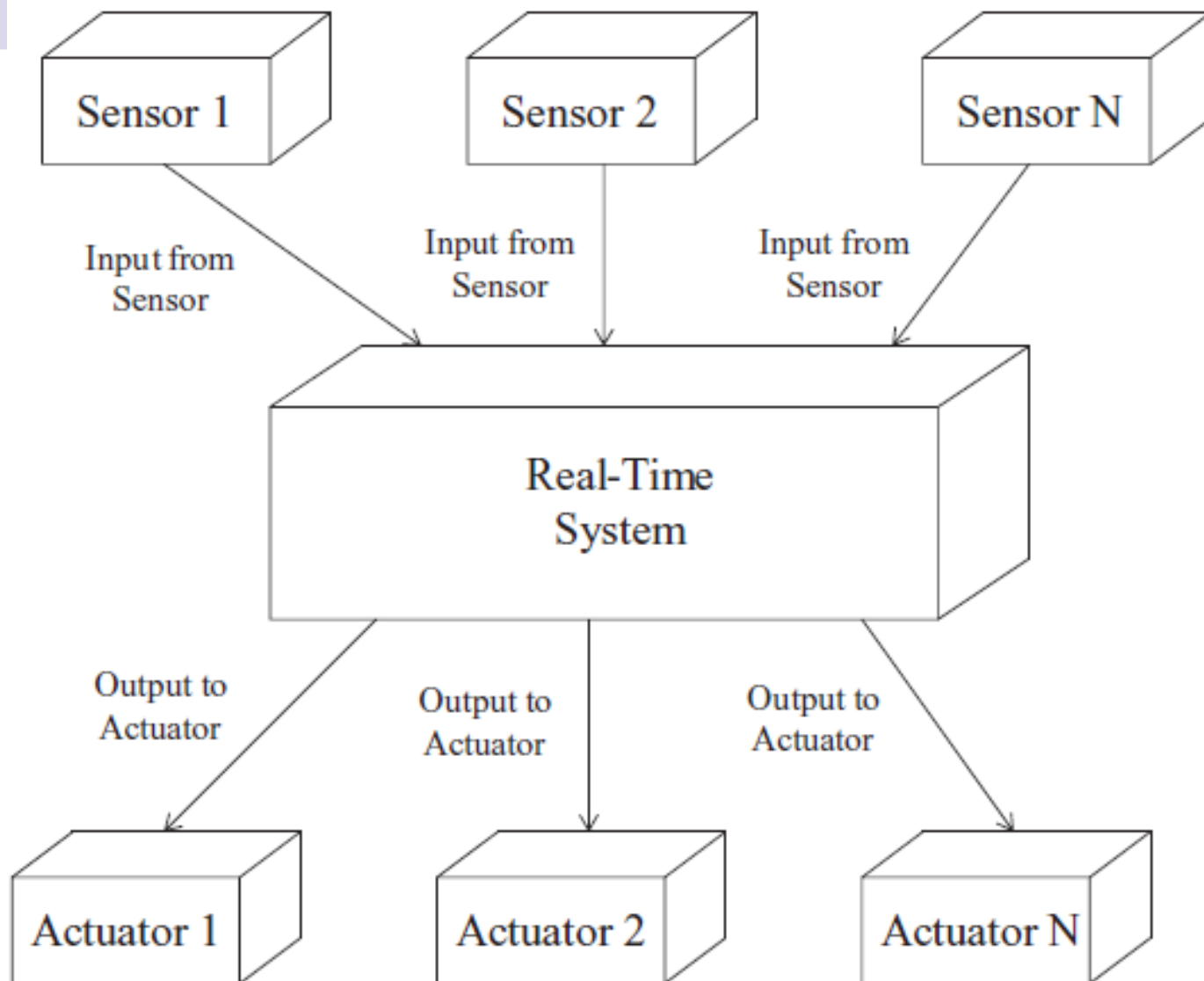
10.1 概述

● 实时系统的特点

- 具有时间约束的并发系统
- 通常指包含了实时应用系统、实时操作系统和实时I/O子系统等整体系统，并拥有与各种传感器和执行器相交互的专用设备驱动程序
- 须处理多个相互独立的输入事件流并产生多个独立的输出
- 事件必须符合系统需求中给出的时间性约束
- 事件到达时间、到达顺序难以预测
- 输入负载可能会随时间发生不可预测的剧烈变化



10.1 概述



10.2 实时软件架构中的控制模式

- 控制模式包括：
 - 集中式控制模式
 - 分布式控制模式
 - 层次化控制模式



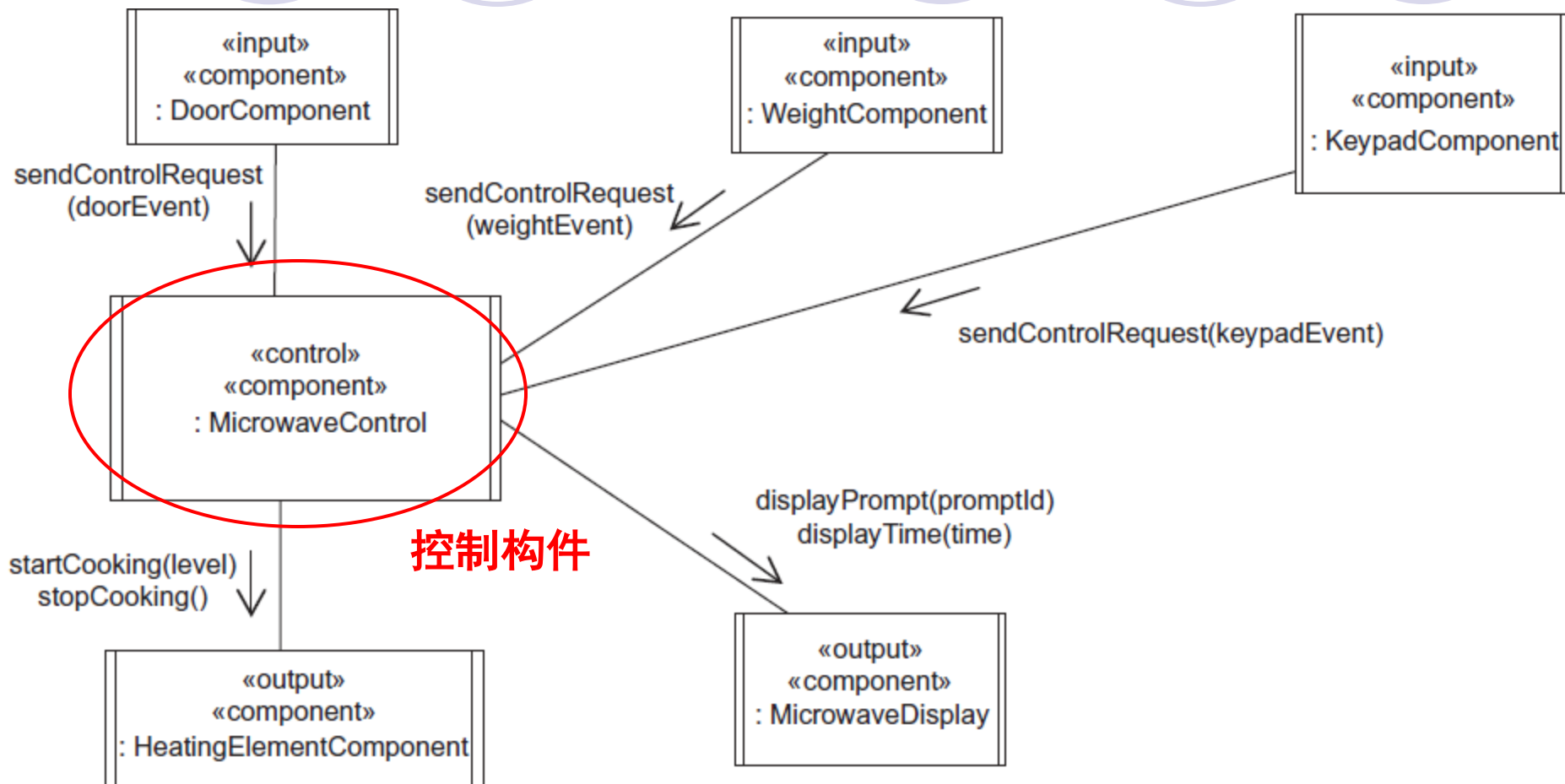
10.2 实时软件架构中的控制模式

- 集中式控制模式

- 仅包含单个控制构件
- 控制构件执行一个状态图以总控全局并管理整个系统的行为顺序
- 控制构件从与其交互的构件那里接收事件，并引发一个或多个状态相关动作
- 控制构件通过动作控制系统中的其他构件
- 例子：微波炉控制系统



10.2 实时软件架构中的控制模式



10.2 实时软件架构中的控制模式

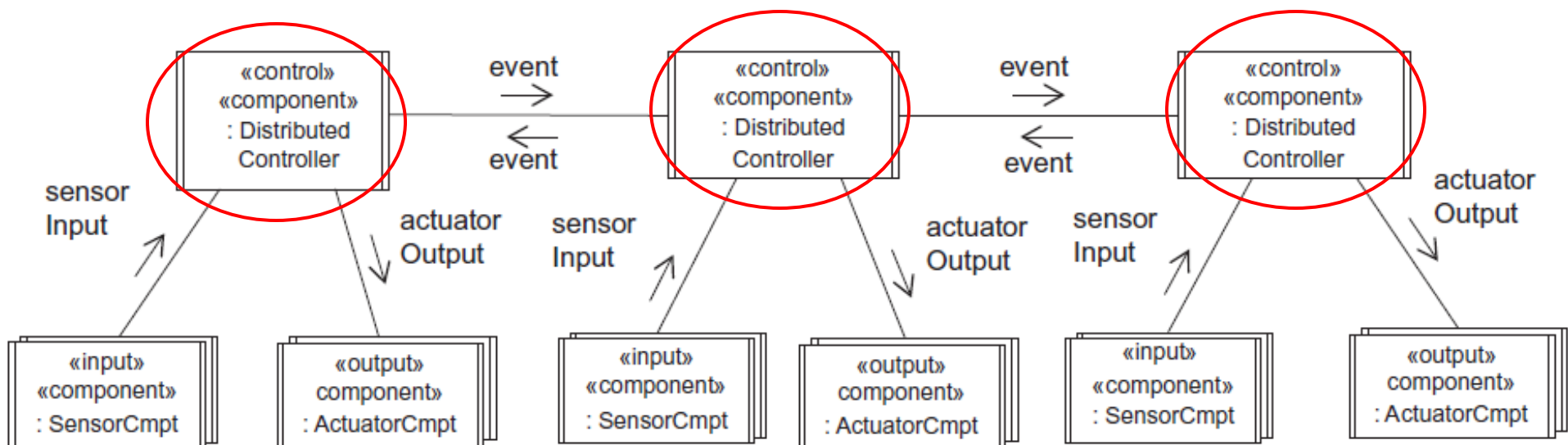
- 分布式控制模式

- 包含多个控制构件
- 每个控制构件执行一个状态图以控制系统的**特定部分**
- 控制分布于各个控制构件中
- 控制构件通过点对点的通信实现重要事件的通知
- 典型架构示意图



10.2 实时软件架构中的控制模式

分布式控制构件



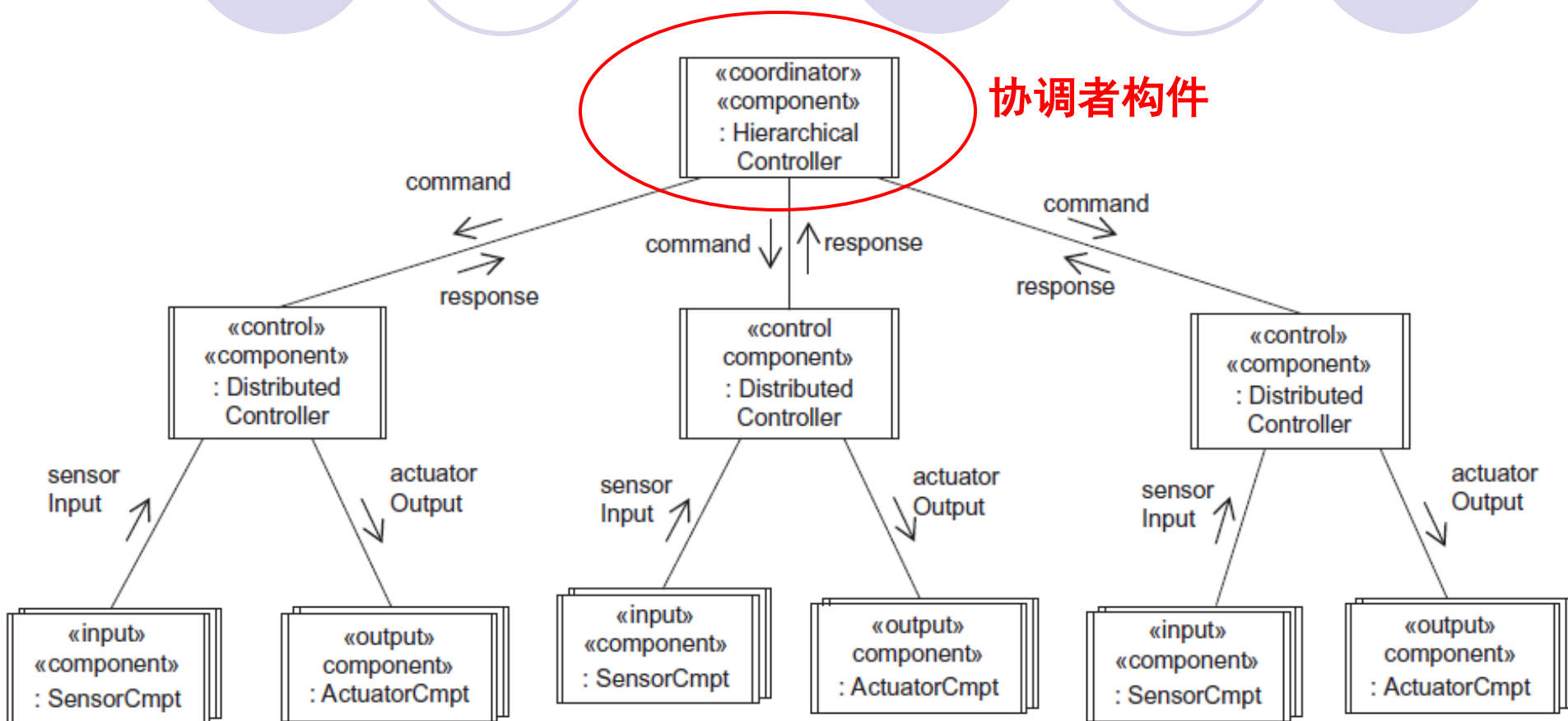
10.2 实时软件架构中的控制模式

● 层次化控制模式

- 包含多个控制构件
- 每个控制构件执行一个状态图以控制系统的 **特定部分**
- 存在一个协调者构件，它协调所有控制构件来完成整个系统的控制
- 协调者构件提供高层控制：
 - 直接与各个控制构件通信并决定各个控制构件的动作
 - 从控制构件接收状态信息
- 典型架构示意图



10.2 实时软件架构中的控制模式



10.3 并发任务

- 并发任务是一个主动对象，指一个进程或线程
 - 在一些系统中，并发任务实现为具有单个线程的进程
 - 也有一些系统，并发任务实现为进程中的单个线程
- 在分析模型中确定对象应该并发执行或者顺序执行，从而被设计为主动或被动对象
- 两类重要的并发任务
 - **I/O任务 & 内部任务**



10.4 I/O任务组织准则

- 通过与I/O任务交互的I/O设备识别任务
- I/O任务包括：
 - 事件驱动
 - 周期性
 - 按需驱动



10.4 I/O任务组织准则

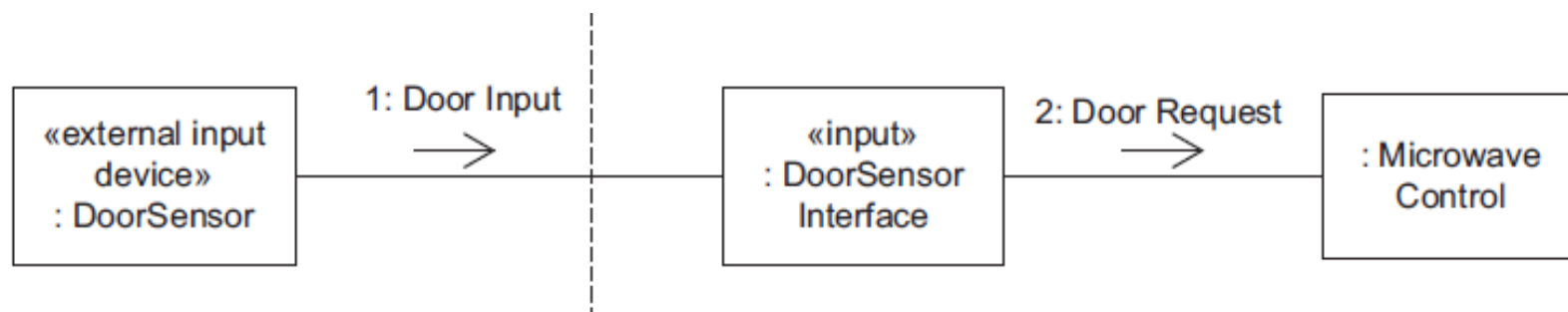
● 事件驱动I/O任务

- 系统需要与事件驱动（中断驱动）的I/O设备交互
- 事件驱动的I/O设备会触发事件驱动I/O任务
- 事件驱动I/O任务接收输入事件（中断），将它传给其他对象处理，随即转而对之后到来的中断做出迅速响应
- 事件驱动I/O任务也可以处理外部系统传来的消息，亦称作：事件驱动代理
- 构造型：<<event driven>>
- 例子：“门传感器接口”

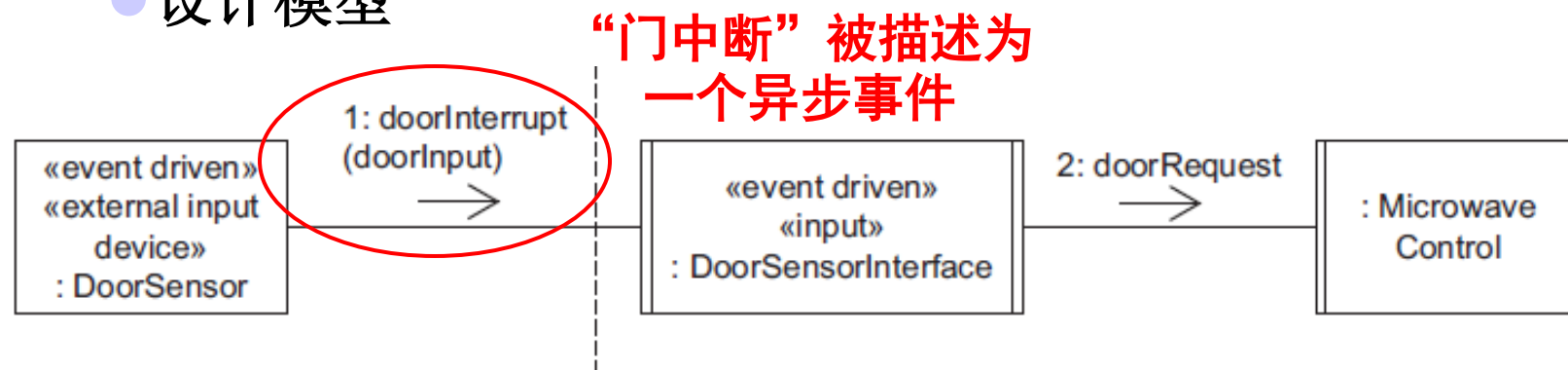


10.4 I/O任务组织准则

● 分析模型



● 设计模型



10.4 I/O任务组织准则

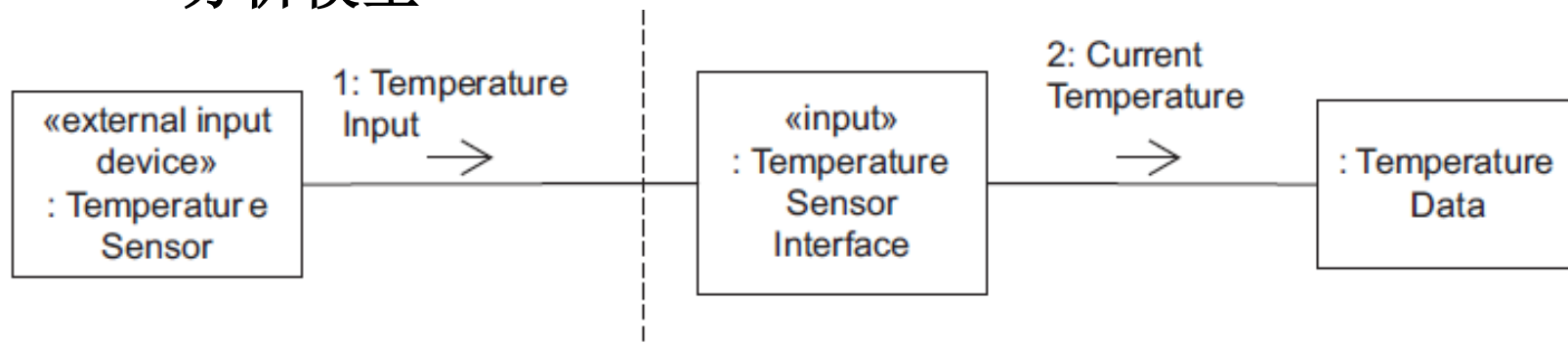
- 周期性I/O任务

- 定期轮询一个 **被动I/O设备**
- 任务的触发是周期性的
- 如：外部计时器周期性地发送计时器事件，从而触发周期性I/O任务
- 周期性I/O任务通常用于需要周期性采样的被动传感器设备
- 构造型：<<periodic>>
- 例子：“温度传感器接口”

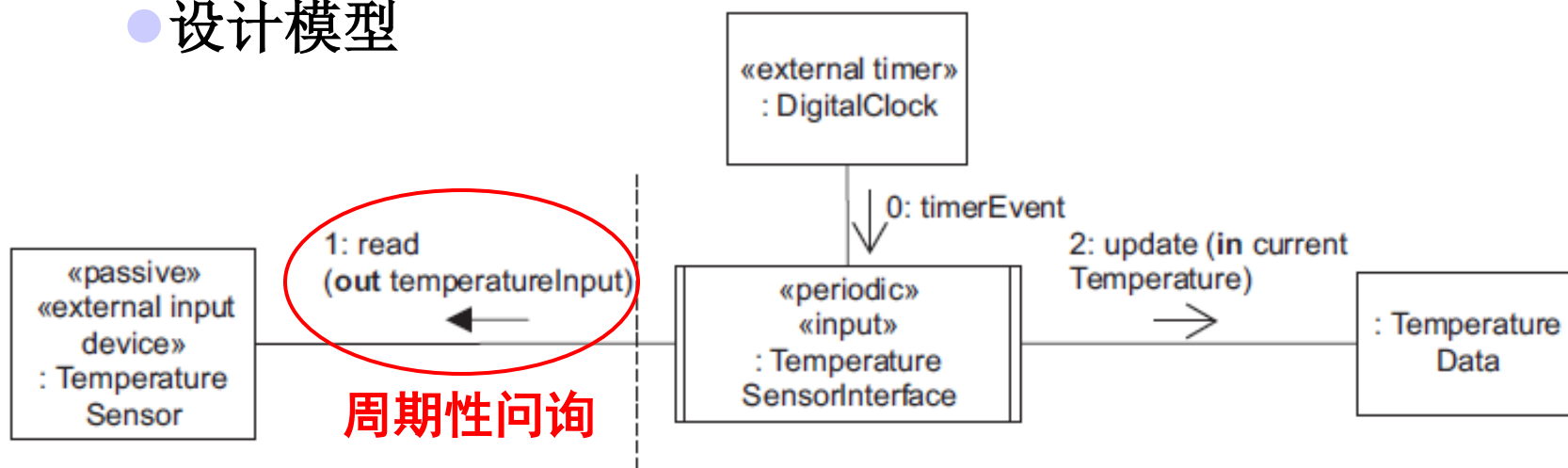


10.4 I/O任务组织准则

● 分析模型



● 设计模型



10.4 I/O任务组织准则

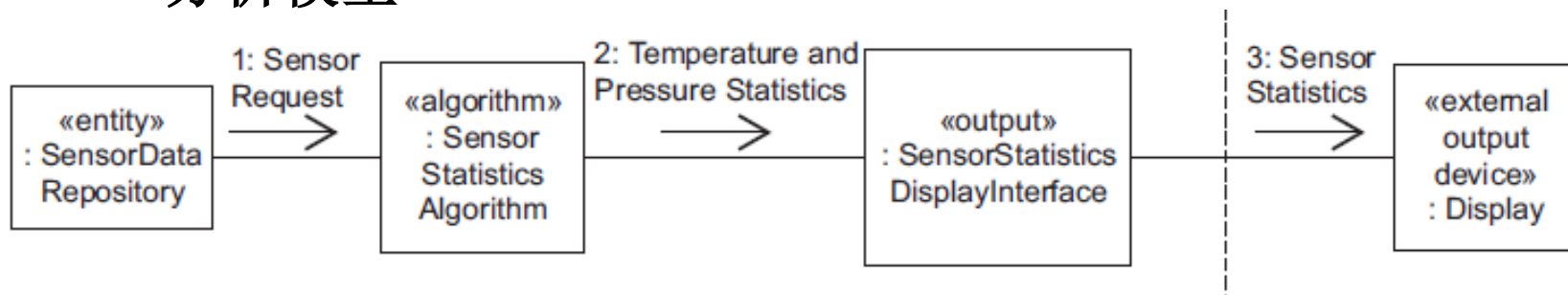
- 按需驱动I/O任务

- 处理不需周期性轮询的 **被动I/O设备**
- 对于输入，需要同时进行接收被动设备的输入以及接收和消费数据的计算性任务
- 对于输出，需要同时进行对设备的输出和产生数据的计算性任务
- 按需驱动I/O任务更常用于处理输出设备
- 构造型：<<demand>>
- 例子：“传感器数据统计显示接口”

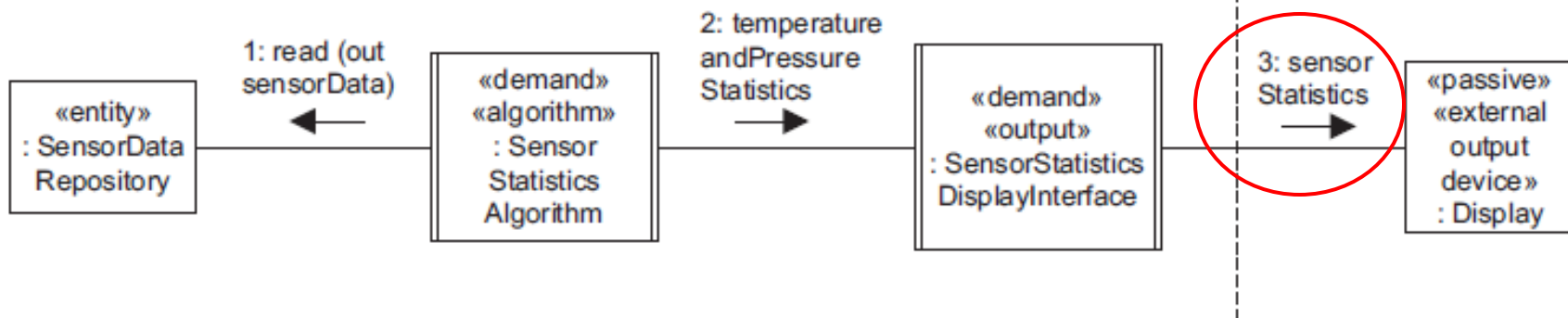


10.4 I/O任务组织准则

● 分析模型



● 设计模型



10.5 内部任务组织准则

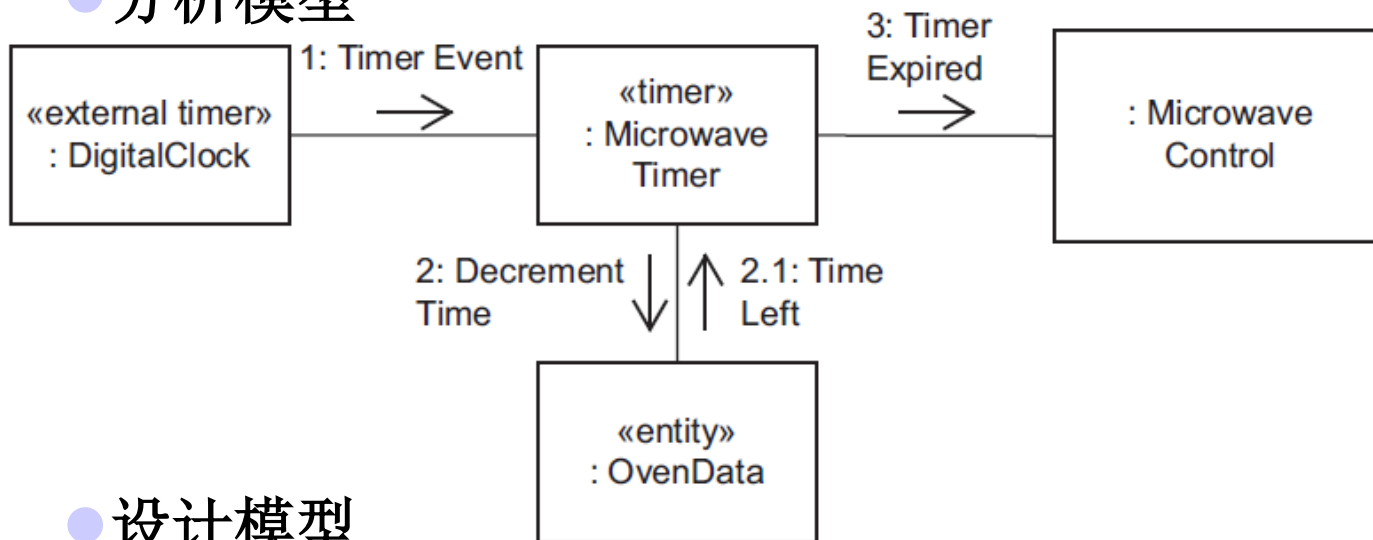
● 周期性任务

- 用来处理 **周期性** 执行的 **活动** 的任务
- 可以执行 **周期性算法**
- 该任务被计时器事件触发并执行周期性活动，随后等待下一个计时器事件
- 构造型：<<periodic>>
- 例子：“微波炉计时器”

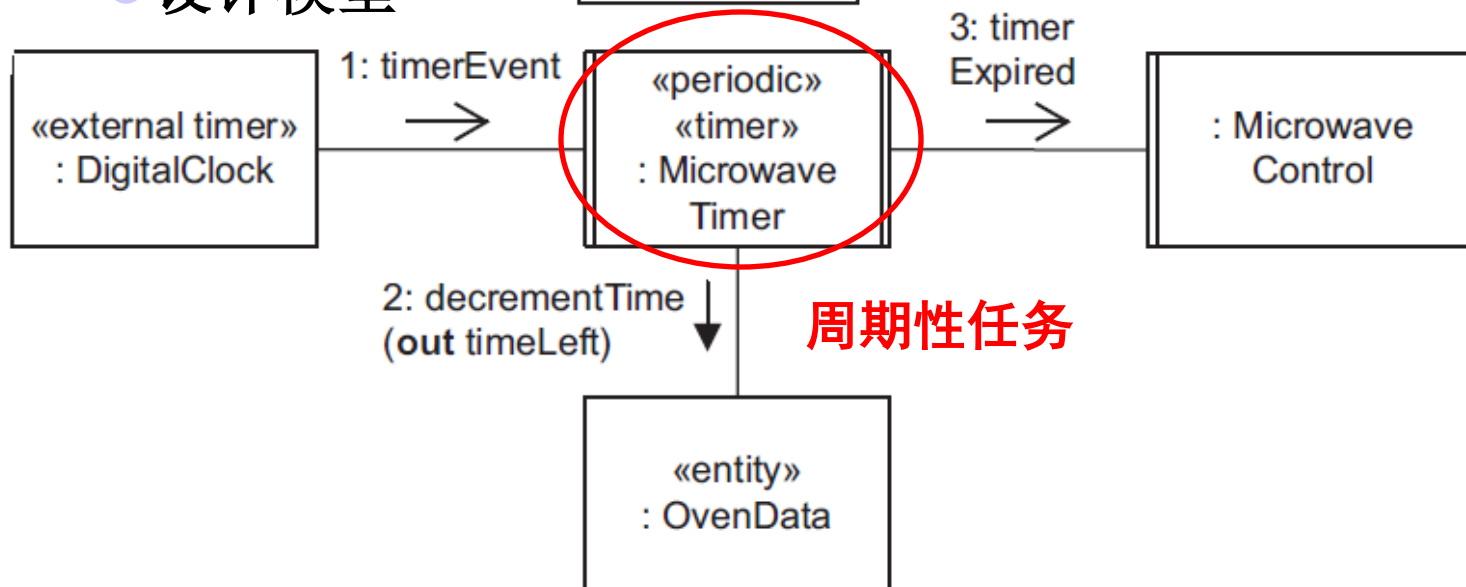


10.5 内部任务组织准则

● 分析模型



● 设计模型



10.5 内部任务组织准则

- 按需驱动任务

- 也被称为 **非周期性任务**
- 它被到达的外部消息或事件所触发，触发后任务开始处理所需请求，随后等待下一个消息或事件
- 可以是 **按需驱动算法**
- 构造型： <<demand>>、<<algorithm>>



10.5 内部任务组织准则

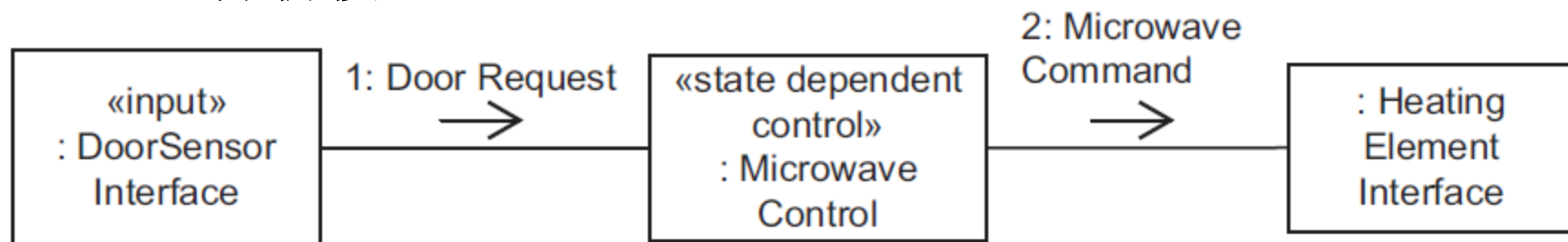
● 控制任务

- 是分析模型中的状态相关的控制对象在设计模型中的实现
- 每个控制任务执行顺序状态图
- 通常是 **按需驱动** 的任务，需要由其他任务发送的消息所触发
- 分析模型中的协调者对象也 被设计成 **协调者任务**，它也是一种控制任务
- 构造型： **<<state-dependent control>>**
- 例子：“微波炉控制”

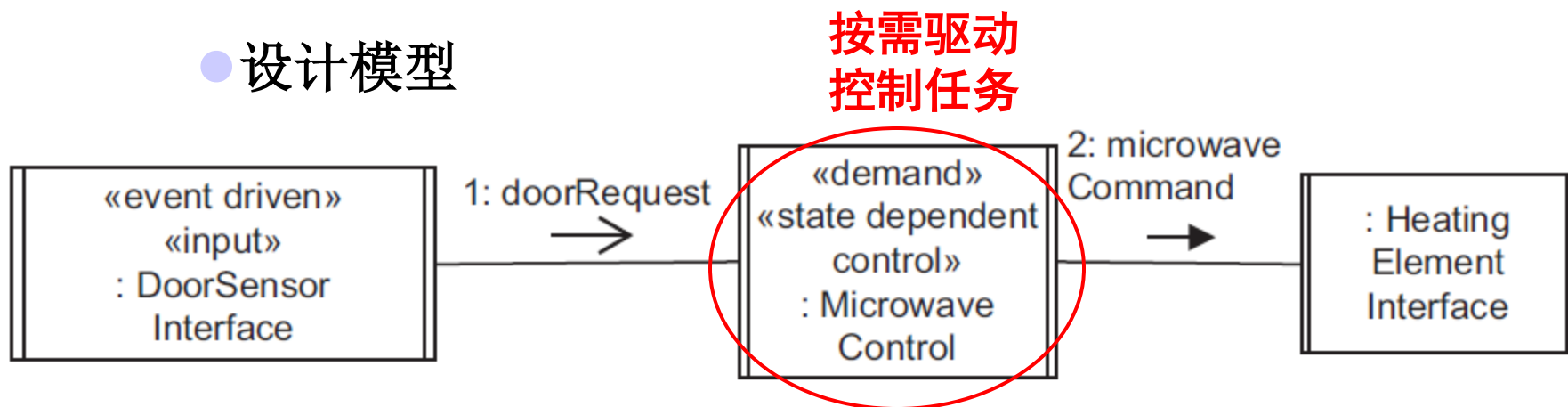


10.5 内部任务组织准则

● 分析模型



● 设计模型



10.5 内部任务组织准则

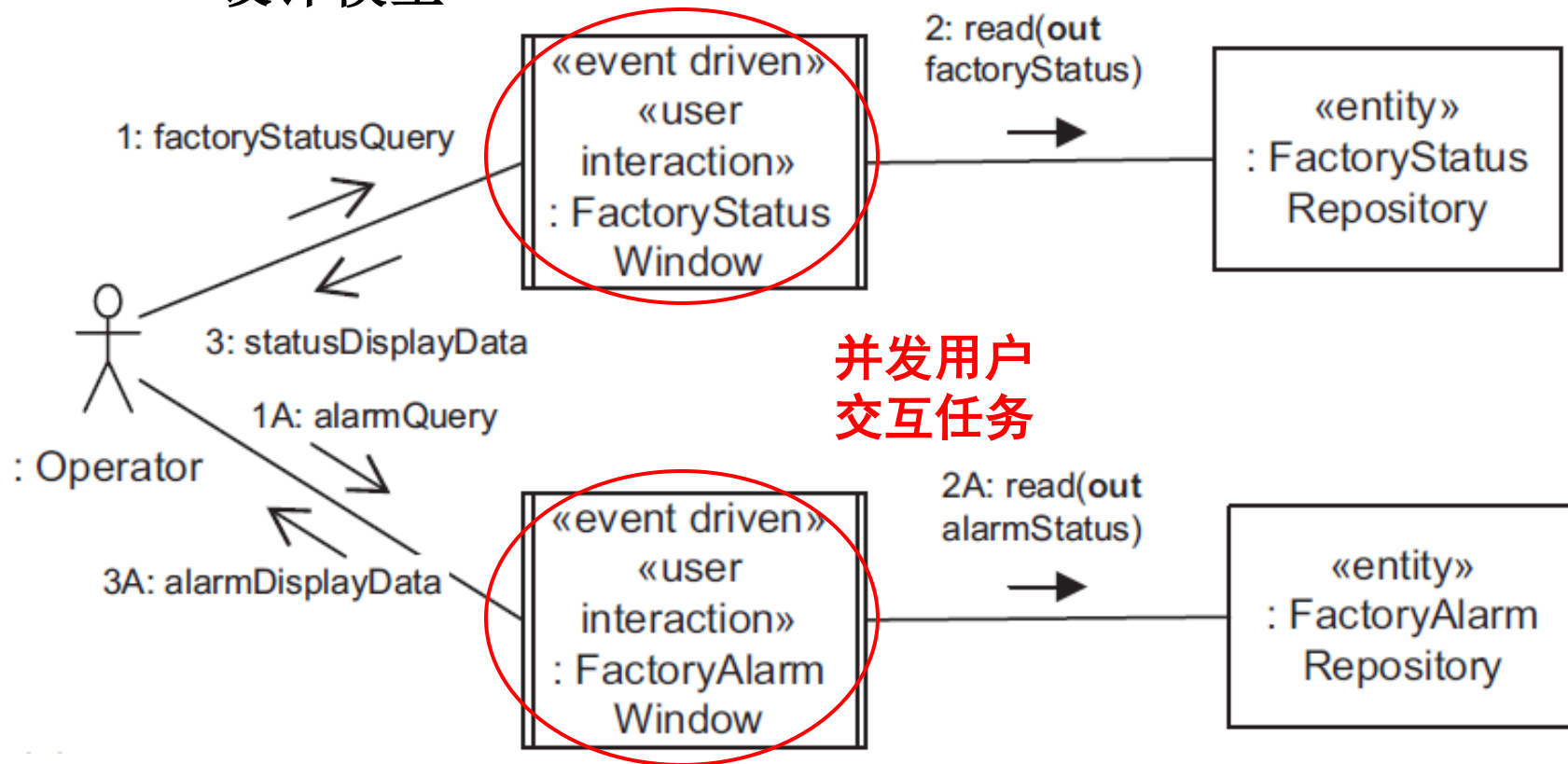
● 用户交互任务

- 分析模型中的用户交互对象被设计成用户交互任务
- 用户交互任务通常是事件驱动的
- 一个用户交互任务通常与多个标准I/O设备相连，由于操作系统为这些设备提供了标准接口，所以无需开发专用的I/O任务去处理它们
- 在多用户操作系统中，是多任务（进程）设计的，所有用户交互任务都并发执行（如，多窗口）
- 构造型<<event driven>>,<<user interaction>>
- 例子：多窗口交互



10.5 内部任务组织准则

● 设计模型



10.6 开发并发任务架构

- 首先确定分析模型对象在设计模型中要被设计为主动对象（即任务）还是被动对象
- 然后按以下顺序应用以上任务组织准则
 - I/O任务
 - 从外部的设备I/O对象来分析
 - 控制任务
 - 分析每个状态相关控制对象和协调者对象
 - 周期性任务
 - 分析内部周期性活动
 - 其他内部任务
 - 由内部事件触发内部任务，构造为按需驱动任务



10.7 任务间的通信和同步

- 任务间的通信可以是异步、同步，包括：
 - 异步消息通信（松耦合）
 - 消息生产者给消费者发送完一个消息后无需等待
 - 带回复的同步消息通信（紧耦合）
 - 消息生产者给消费者发送完一个消息后会等待回复
 - 不带回复的同步消息通信（紧耦合）
 - 消息生产者给消费者发送完一个消息后会等待消息被消费者接收，一旦消费者接收消息则释放生产者



10.7 任务间的通信和同步

- 事件同步机制，包括：
 - 外部事件同步
 - 事件来自外部对象，通常是外部I/O设备的中断
 - 来自设备的一个事件信息输入，跟随一个来自任务的读取操作
 - 计时器事件同步
 - 一个任务的周期性触发
 - 内部事件同步
 - 源任务和目标任务之间的内部同步
 - 任务之间不进行数据通信
 - 源任务发出事件信号，目标任务在信号到达前保持等待并挂起，直到接收到事件信号为止

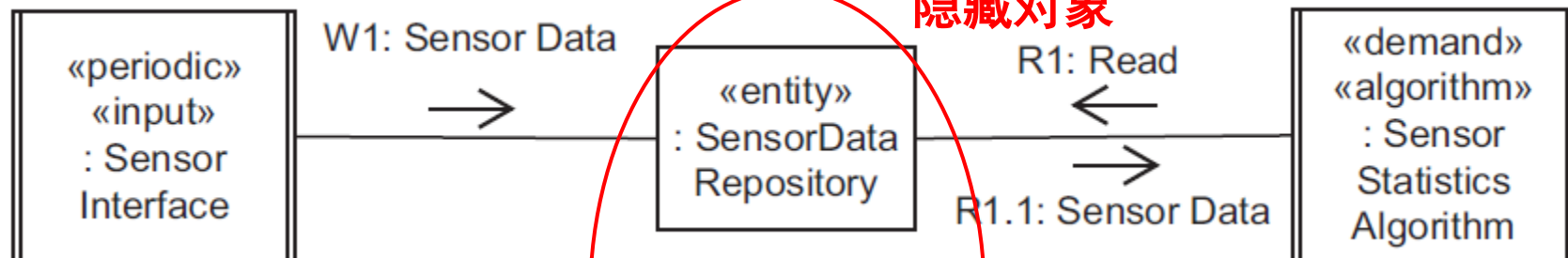


10.7 任务间的通信和同步

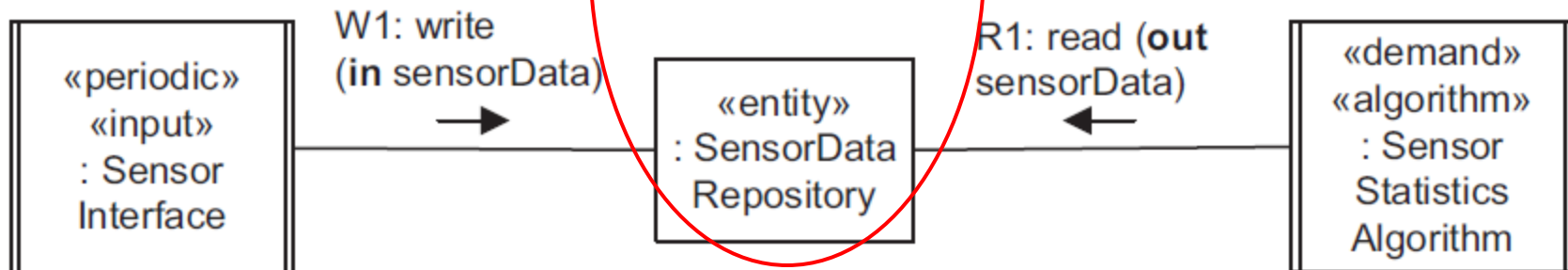
● 信息隐藏对象上的任务交互

- 任务之间也可以通过被动信息隐藏对象交换信息
- 例子：传感器数据存储

● 分析模型



● 设计模型



10.8 任务接口和任务行为规约

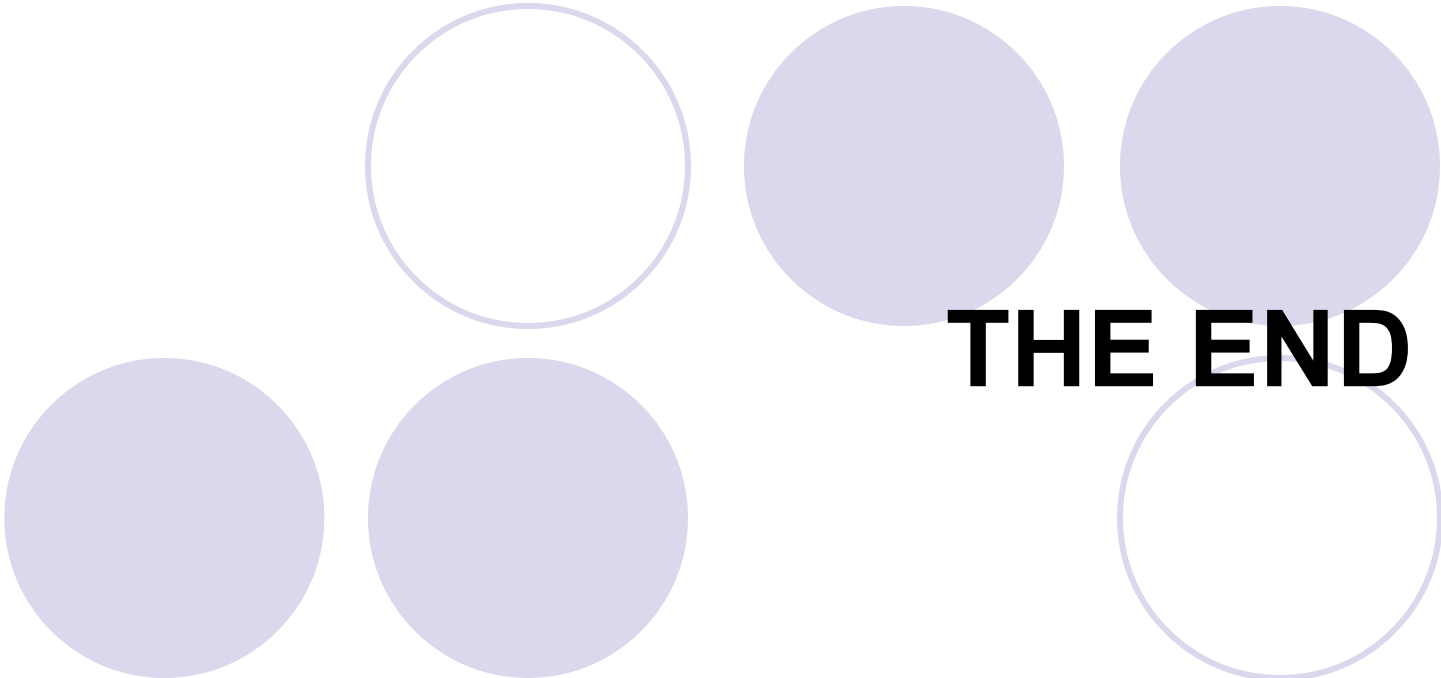
- 任务接口定义了任务之间的交互方式
- 任务接口规约（**TIS**）是类接口规约的扩展
- 增加了以下内容：
 - 任务结构
 - 描述了如何使用任务组织准则构造任务
 - 时序特征
 - 表示任务触发频率以及估计的执行时间，用于实时调度
 - 相对优先级
 - 任务执行的优先级，用于调度
 - 错误检测



10.8 任务接口和任务行为规约

- 任务行为规约（**TBS**）描述任务事件的顺序逻辑，即任务如何对接收到的消息或输入事件做出响应



Five large circles are arranged in a pentagonal pattern. The top circle is an outline. The bottom-left and bottom-right circles are solid light purple. The top-right and bottom-middle circles are solid light purple. The top-right circle is partially obscured by the text 'THE END'.

THE END